University of Bergen - Department of Informatics

Top Trading Cycles in a practical setting

Author: Lars Møen Haukland

Supervisor: Fredrik Manne

UNIVERSITETET I BERGEN
Fakultet for naturvitenskap og teknologi

June, 2026

Abstract

This thesis explores the Top Trading Cycles algorithm in the practical setting of the allocations of GPs, General Practitioner, in Norway for people wanting to switch or get a GP. It explores how different focus of *good* results can change both the runtime and results, and how to make the algorithm run as efficiently as possible. This thesis proposes an implementation of the Top Trading Cycles that can reduce the waitlists for GPs in norway by over 80%.

Acknowledgements

I would like to thank my supervisor Fredrik Manne for his guidance and support throughout this project. I would also like to thank Juni Weisteen Bjerde for input and help during the project. In addition Juni and Mathias Bertntert together theorised the exact algorithms so that I could implement them.

Contents

1. Introduction

The Norwegian general practitioner (GP) system, started in 2001, has aim to serve the Norwegian population with a stable GP. A GP can follow his or hers patients over time, creating a trustful and safe relationship [1]. Today this system works as intended, and GPs handle most of the populations needs. As of December 2025 there are 5720 GPs in Norway serving a population of 5.6 million. The number of patients each GP has varies but on average each GP has about a thousand patients [2].

One inefficiency in the Norwegian GP system is when a person wants to switch from one GP to another. If the GP they want to switch to does not have any more capacity for new patients, the person has to sign up in a queue and wait for a free slot to open up. In December 2025 the number of open GP lists was 1340, indicating that 77% of all GPs had no extra capacity [3]. In a more densely populated city such as Bergen, the numbers were even worse. Where only 7 out of 268 GPs had available capacity for new patients. Thus 97% of the GPs in Bergen had no extra capacity [3].

When a person wants to switch to a GP that has no excess capacity they must sign up and then wait in line. Open slots are allocated on a first come first serve basis. In December 2025 there were about 300,000 people waiting for GPs in Norway [2]. As there were only 5720 GPs each one has on average more than 50 people on his or hers waiting list. Since there were a relatively large number of people who are waiting to either switch or to be allocated a GP it follows that it is likely that there are cycles, in that a person P_A wants GP D_A which currently has P_B as a patient, but P_B wants to switch to GP D_B who currently has P_A as a patient. In this case patient P_A ends up waiting for an open slot with P_B's GP and vice versa, but logically instead of waiting P_A and P_B could just swap GPs. This can be generalized to longer "waiting cycles" that could all be serviced by allowing for simultaneous switching.

In this thesis, we study how such cycles can be resolved and propose different strategies for resolving these. In particular we investigate the use of cycle detecting algorithms to find optimal switches between patients and show that this can lead to a substantial reduction in the number of people on waiting lists by running these algorithms periodically. Our algorithms are based on existing cycle cancelling and TTC algorithms. While the focus is always on helping as many patients as possible we study two different variations of this problem, when there is a priority among those waiting and when there is no priority. We have developed and tested optimal algorithms as well as heuristics for these cases and evaluated them for solution quality as well as speed.

In addition we compare our result with those of the existing algorithms. We note that our solutions have applications to other types of reallocation problems where users want to switch their given assignment. This could for instance be in reallocation of houses, kidney donors or students switching schools.

1.1. Thesis Outline

The remainder of this thesis is organized as follows:

- **Section 2** provides background on the TTC mechanism, cycle cancelling algorithms and how these methods have been used on similar problems.

- **Section 3** formally defines the computational problems arising from TTC as variants of cycle cover problems on directed graphs.
- **Section 4** describes the algorithmic strategies employed in our implementations.
- **Section 5** presents experimental results comparing different priority strategies.
- **Section 6** concludes and discusses future work.

2. Background

Problems like the GP allocation problem have been studied for quite some time with a number of variations. In addition there are different perspectives on how to solve these problems. For instance in economics the focus is often on the fairness of an assignment, trying to optain a stable and strategy proof assignment. In this chapter we describe similar problems to the GP allocation problem, describe the Top Trading Cycles algorithm (TTC) and the use of this by Huitfeldt et al for the GP allocation problem [4]. Finally we describe cycle cancelling, an optimization technique that our exact algorithms are based on.

2.1. Related problems

There are many different variations of assignment problems. In a typical setting there are agents and objects where the agents want to obtain particular objects. In the GP allocation problem the patients are the agents and GPs are the objects. The typical assignment problems are one-to-one meaning that one agent is assigned to at most one object and an object can only be "owned" by one agent. In addition we also have many-to-one assignment problems, like the GP allocation problem where one agent can only be assigned to one object but one object can be "owned" by more than one agent. The terminology that a patient "owns" a GP means that the patient has that GP as his or hers current GP. It might be misleading to say that our patients "own" doctors but this makes more sense in other problems that we will describe, owning a doctor in our case means having that doctor as a current doctor.

2.1.1. One-to-one assignment problems

Recall that in a one-to-one assignment problem each agent is assigned to at most one object and each object can be "owned" by at most one agent. We look at two such problems, the Housing Marked and the Kidney Exchange. Both differ from the GP allocation problem since a GP can be owned by many patients at once, but can be solved using the same algorithms.

2.1.1.1. Housing Market

The Housing Market model introduced in Shapley and Scarf [5] describes a model with traders (agents) and indivisible goods (objects). These goods can be houses, making it a housing market. Each agent already has one house but may want to switch. Every agent has a preference list ordering every house in order of preference, Shapley and Scarf combine all these preference lists to make a preference matrix [5].

It is in this article from Shapley and Scarf [5] that they also introduce the Top Trading Cycles algorithm and describe how it is fitting for models like the Housing Market. We also see similarities to the GP allocation problem in that we have agents already owning an object but wanting to switch. However two differences are that a GP can have multiple patients while a house can only be owned by one agent, and also that in the Housing market each agent has a complete preference list of objects while in the GP assignment problem each agent has only one preferred GP.

2.1.1.2. Kidney exchange

The Kidney Exchange problem describes an issue when trying to find compatible kidney donors. It describes the case where we have pairs of patients and donors, but the patient is

incompatible with its donor's kidney. We then need to find some way to swap around the donors to compatible patients. Either with pairwise swapping or through longer cycles. If we visualize the problem as a directed graph, each vertex is a patient and donor pair. An edge from pair A to pair B means that donor A is compatible with patient B. Now cycles can form as in Figure 1 where we have a cycle of length three $A \to B \to C$. This means that the donor in pair A can be matched with the patient in pair B and so on, assuring that each patient in the cycle is assigned a compatible donor.

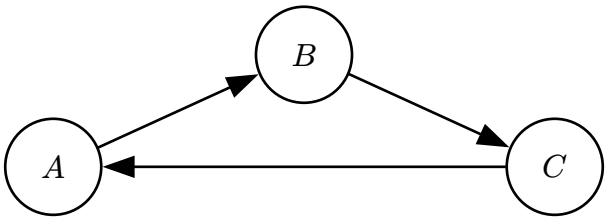

Figure 1: An example graph for the kidney exchange problem.

Abraham, Blum and Sandholm [6] show that, with unbounded cycle length, a maximum-weight exchange can be found in polynomial time [7]. An issue often arising with unrestricted cycle length is if a donor suddenly refuses to donate. If this happens after their patient has received a kidney then a patient is left without a donor and cannot exchange later. Because of this the problem is often considered with a restricted cycle length to allow all operations to be executed at the same time. This way no donor can refuse in the middle. For each pair in a cycle you need 2 operations, for a cycle of length k you need $2k$ operations at the same time. It is for this reason that the Kidney Exchange problem often is considered with a max cycle length k, as the number of operating rooms in a state/city is limited. Finding a maximum-cardinality exchange with cycles of length at most k, for any fixed $k \geq 3$, is an NP-hard computational problem [6], [7].

The Kidney Exchange problem and the GP allocation problem have much in common. In both problems agents already have an object, but want to switch for another. We need to find cycles to exchange the objects in such a manner that as many patients as possible are satisfied. The key difference is that only one donor can be "owned" by one patient while in the GP allocation problem a doctor can be "owned" by multiple patients.

2.1.2. Many-to-one assignment problems

In a many-to-one assignment problem each agent is still assigned to at most one object, but an object can be "owned" by more than one agent. This is the same structure as the GP allocation problem, where a GP can have many patients. We look at one such problem, the College Admissions problem.

2.1.2.1. The College Admissions problem

The College Admissions problem, introduced by Gale and Shapley [8], describes the problem of assigning applicants to colleges. Each college has a certain quota, i.e. a maximum number of applicants that can be admitted and each applicant ranks the colleges in order of preference. An applicant does not have to rank every college and can skip colleges he or she does not want to attend [8].

The problem is then to find a stable assignment, meaning that there does not exist a case where there are two applicants α and β who are assigned to colleges A and B, although β prefers A to B and A prefers β to α. Such a pair would break the assignment, since both would rather be matched to each other.

There can be many stable assignments for the same preferences, so there is not one single best assignment. Among all stable assignments there is one that is best for every applicant at the same time, and one that is best for every college at the same time. Gale and Shapley [8] introduce an algorithm called deferred-acceptance that finds a stable assignment in polynomial time [8]. The algorithm has one side propose and the other side accept or reject. The side that proposes ends up with the assignment that is best for them and worst for the other side.

There are some similarities between the College Admissions problem and the GP assignment problem. Like how applicants prefer some colleges while patients prefer a GP. One key difference is that patients, only prefer one GP. Another difference is that patients already have existing GPs, while applicants do not already have a college they want to switch from.

2.2. Top Trading Cycles

The Top Trading Cycles, or TTC, is an algorithm that finds cycles of agents that can all trade their objects at the same time. We look at it in two forms. First we explain the classical TTC, which works on the Housing Market problem. This is the original version and we explain it to show where the method comes from. Then we explain the version made by Huitfeldt et al. for the GP allocation problem. It is this version that inspired our work, and it is the one we later compare our algorithms against.

2.2.1. Classical TTC

The Top Trading Cycles algorithm (TTC), developed by Gale and published by Shapley and Scarf [5], is an algorithm to find a re-allocation of objects, or goods, without using money. As mentioned it was introduced in an article together with the Housing Market problem. The algorithm finds a re-allocation of houses to traders, such that all mutually-beneficial exchanges have been realized [9].

TTC works on a directed graph, for the following implementation, on the Housing Market problem, each agent is a node and edges point from agents to other agents. The algorithm then does the following [9]:

1. Query each agent for its "top" (most preferred) house.
2. Insert a directed edge from each agent i to the agent, denoted $\mathrm{Top}(i)$, that holds the most desired house of i.
3. Find a cycle (which is guaranteed to exist) and execute the trades in that cycle. Next, remove all involved agents from the graph.
4. If there are remaining agents, repeat from step 1.

Note that a cycle is guaranteed to exist if there are agents still left. This is because of the pidgeonhole principle, since each agent has one outgoing edge we have to have a cycle. The cycle can also be of length 1 in which case an agent's top house is its own, then we treat it as a normal cycle and remove the agent from the graph. For this classical TTC implementation to work we need to remember that agents have a strict preferences. Each agent has a complete

list of all houses ranked in order of preference. Because of this as houses get re-allocated agents are bound to either find a trade cycle with others or end up having their $\text{Top}(i)$ house as their own.

Consider the example of 3 agents, A, B and C, each has a house and have a strict preference list. Lets go through how TTC would handle this:
The preference lists are
- $A = [B, C, A]$
- $B = [C, A, B]$
- $C = [B, C, A]$

If we make the graph where each agent point to their top we get Figure 2.

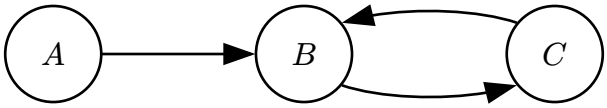

Figure 2: An example Housing Market for TTC.

We have the cycle $B \to C$ so we execute the trades, remove agents from the graph and create the edges again. The last remaining node is A and as B and C are not in the graph, $\text{Top}(i)$ of A is now A.

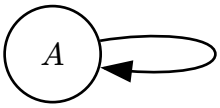

Figure 3: An example Housing Market for TTC, A's $\text{Top}(i)$ is itself.

We execute this trade and are left with no more agents, making TTC terminate.

Roth proved that TTC is strategy proof, meaning all agents are motivated to prefer their true preferences [10]. TTC has also satisfies properties of Individual rationality, that an agent is at least as well of by participating, and Pareto efficiency which is that the objects are allocated such that no agent can improve without worsening another agent.

Now using TTC for the GP assignment problem is challenging as patients do not have complete strict preference lists, they only have one preferred doctor, and a doctor can be "owned" by multiple patients. This makes the $\text{Top}(i)$ give out multiple patients, as the doctor that patient i prefers is "owned" by multiple patients. How should we choose between these? This is what Huitfeldt et. al has written about and how their implementation satisfies the same properties of the classical TTC. Later we run the TTC made for the GP assignment problem and compare with our algorithms. First we explain it and how it is different from normal TTC.

2.2.2. GP assignment problem TTC

In the paper from Huitfeldt et al. they tackle a more complex dataset as they use real patient and doctor data from Norway. In their model they have doctors with available capacity and patients switching to these GPs can be allocated that GP without an exchange. This is why in each of the two TTC implementations they first run a waitlist algorithm that goes through all GP with available capacity and assigns the highest priority to that GPs panels until there are

no more patients waiting on GPs with available capacity [4]. Following we explain Huitfeldt et al. TTC implementation without regard to capacities to doctors as that is how we tackle the problem.

Now the algorithm is as follows [4]:

1. Each patient points to their preferred GP, if the preferred GP is not in the graph the patient points to its current GP. Each GP points to the patient in their panel with highest priority.
2. Find a cycle in the graph, remove patients part of that cycle. If a GP has no more patients that are in the graph it is removed from the graph.
3. Repeat step 1 until there are no more patients

Like the classical TTC we are guaranteed to have a cycle because of the pidgeonhole principle, since if a patient does not get their preferred GP they end up pointing to their own GP making a cycle.

2.3. Cycle cancelling

Cycle cancelling is a technique used in flow and circulation problems to find a minimum cost solution. First we explain some of the terms needed for cycle cancelling then we go onto what cycle cancelling is. We use definitions from Ahuja, Magnantti and Orlin [11].

2.3.1. The minimum cost circulation problem

Let $G = (V, A)$ be a directed multigraph with n vertices and m edges. Each edge is a triple (v, w, i) where v is the tail, w is the head and i is an index. The pair (v, w) gives the endpoints of the edge. The index i separates edges that have the same endpoints, so G can have parallel edges. This means an edge is identified by the full triple and not just by its endpoints.

The multigraph G is a circulation network if each edge (v, w, i) has a capacity $u(v, w, i) \geq 0$ and a cost $c(v, w, i) \in \mathbb{Z}$. For a vertex $w \in V$ we let $\text{in}(w) = \{(v, w, i) \in A\}$ be the edges whose head is w, and $\text{out}(w) = \{(w, v, i) \in A\}$ the edges whose tail is w.

Definition 2.3.1.1 (Circulation): A circulation is a function f that assigns a value $f(v, w, i)$ to each edge and satisfies the capacity constraints

$$0 \leq f(v, w, i) \leq u(v, w, i) \qquad \forall (v, w, i) \in A \tag{1}$$

and the conservation constraints

$$\sum_{(u,w,i) \in \text{ in}(w)} f(u, w, i) = \sum_{(w,v,j) \in \text{ out}(w)} f(w, v, j) \qquad \forall w \in V. \tag{2}$$

The conservation constraint says that at each vertex the flow coming in equals the flow going out. No flow enters or leaves G from the outside, so all flow circulates around the network. The cost of a circulation f is

$$\text{cost}(f) = \sum_{(v,w,i) \in A} c(v, w, i) f(v, w, i). \tag{3}$$

Definition 2.3.1.2 (Minimum cost circulation problem): Given a circulation network G, the minimum cost circulation problem is to find a circulation f with minimum cost.

In our problem each edge is a possible patient switch, and we want a circulation that does as many switches as possible. We get this by setting the cost of every edge to -1. Then $\text{cost}(f) = -\sum_{(v,w,i)\in A} f(v,w,i)$, which is the negative of the total flow. So a circulation with minimum cost is the same as a circulation with the most flow. We do not treat the amount of flow as a separate goal. It is already part of the cost, and the problem stays a normal minimum cost circulation problem.

An example circulation network is shown in Figure 4. For simplicity this example is not a multigraph. Each edge is labeled "x,y", meaning the edge has cost x and capacity y, and as explained above every edge has cost -1. The network has three feasible circulations: no flow on any edge, the cycle $d \to c \to b \to d$, and the cycle $a \to b \to d \to c \to a$. Their costs are 0, -3 and -4. The longer cycle is the optimal circulation since it has the most flow and the lowest cost.

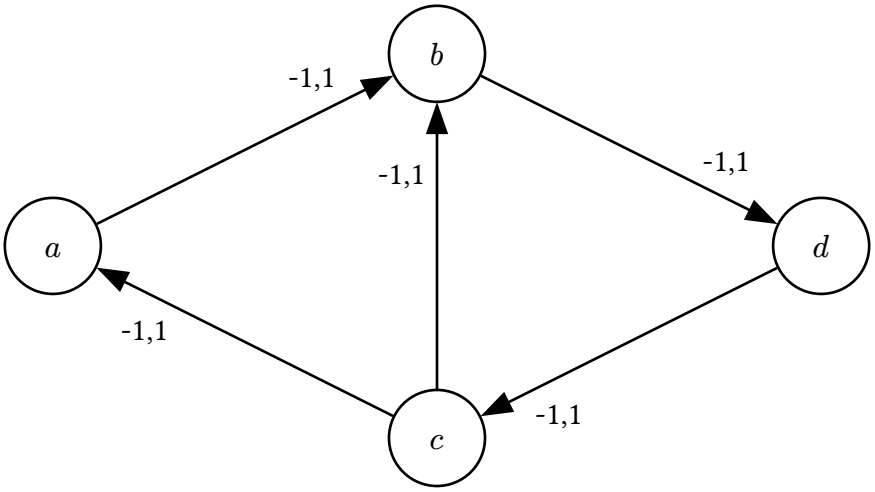

Figure 4: An example circulation network.

2.3.2. The residual network

The residual network shows what capacity is left after a circulation, and what flow can be undone. Given a circulation network G and a circulation f, we build the residual network $G(f)$ edge by edge. Each edge $(v,w,i) \in A$ gives two residual edges.

- A forward edge (v,w,i) with cost $c(v,w,i)$ and residual capacity $u_f(v,w,i) = u(v,w,i) - f(v,w,i)$.
- A backward edge (w,v,i) with cost $-c(v,w,i)$ and residual capacity $u_f(w,v,i) = f(v,w,i)$.

A residual edge is in $G(f)$ only if its residual capacity is greater than 0. An edge with residual capacity 0 is saturated. We build the residual edges from each edge (v,w,i) and not from the pair (v,w), so the index i is kept on both residual edges. This means $G(f)$ is also a directed multigraph and every residual edge is still uniquely identified.

A forward edge has the same cost as its original edge, and pushing flow on it increases f on that edge. A backward edge has the negated cost, and pushing flow on it decreases f on the original edge. So a backward edge undoes earlier flow. A backward edge is only in $G(f)$ up to the flow on its original edge, so we can only undo flow when there is flow to undo.

A negative cost cycle in $G(f)$ is a directed cycle of residual edges where the costs sum to a negative number. The following theorem from Ahuja, Magnanti and Orlin [11] tells us when a circulation is optimal.

Theorem 2.3.1 (Negative Cycle Optimality Theorem): A feasible circulation f^* is an optimal solution of the minimum cost circulation problem if and only if the residual network $G(f^*)$ contains no negative cost directed cycle.

Note that the theorem does not say every circulation with flow is suboptimal. If we push flow around a cycle of cost -1 edges we do create backward edges of cost $+1$, but these form a positive cost cycle and not a negative one. A negative cost cycle is instead an improvement we can still make. It can use forward edges where we add more switches, and backward edges that reroute flow we already committed. When there is no such cycle left, there is no improvement left, and f is optimal.

2.3.3. Algorithm

Using the Negative Cycle Optimality Theorem we can now formalize the cycle cancelling algorithm. The idea is simple. We start with some feasible circulation, and while the residual network has a negative cost cycle we cancel that cycle. The theorem tells us that when there are no negative cost cycles left, the circulation is optimal.

Note that the starting circulation can be 0 on all edges. Sometimes a circulation problem has lower bounds that make this an invalid starting circulation. For cases with lower bounds on edges, a starting circulation can be computed using any max-flow algorithm. In our problem all lower bounds are 0, so the zero circulation is always feasible and we can use it as the start.

```
1 Start with any feasible circulation f, this can be 0
2 while ∃ negative residual cycle c
3 | Cancel c: ∀a ∈ c : f(a) := f(a) + capacity(c)
```

When we cancel a cycle we push flow equal to $\text{capacity}(c)$ along each residual edge in the cycle. Here $\text{capacity}(c)$ is the smallest residual capacity of any edge in c. Recall that the residual network has two kinds of edges. A forward edge (v, w, i) comes from an edge with leftover capacity, and a backward edge (w, v, i) comes from an edge that already has flow on it.

How we push flow depends on the type of edge. When the cycle uses a forward edge (v, w, i) we increase the flow $f(v, w, i)$ on that edge. When the cycle uses a backward edge (w, v, i) we decrease the flow $f(v, w, i)$ on the original edge. This is how the algorithm can undo earlier

flow. A backward edge is in the residual network only up to the flow on its original edge, so we can only undo flow that is actually there.

After we cancel a cycle the residual network changes. An edge that we pushed flow on has less leftover capacity, and its forward edge can become saturated. At the same time its backward edge gets more residual capacity, since there is now more flow that can be undone. This is why a later cycle can push flow back on a backward edge and undo a push we made before.

2.3.4. Runtime

Finding the negative residual cycles can be done with Bellman-Ford in $O(nm)$ time [12]. At each negative residual cycle that is cancelled the total cost of the circulation decreases by at least 1. The cost is bounded below by $-mCU$ where C is the max cost and U is the max capacity of any edge. It follows that the number of iterations is bounded by $O(mCU)$, and then the runtime is $O(nm^2CU)$. Note that this runtime is pseudo-polynomial since it depends largely on the size of C and U.

Goldberg and Tarjan proved that a variant of the cycle cancelling algorithm called Minimum Mean Cycle Cancelling has a strongly polynomial bound on its runtime [13]. But we will later show how for our problem we still have a polynomial algorithm using the classical Cycle Cancelling algorithm.

3. Problem Formalization

In this chapter we define the GP allocation problem and also some variations of it. In addition we go through how to construct a graph given a GP allocation problem and finally how we can reduce the GP allocation problem to the Cycle Cover problem in directed graphs.

3.1. The GP allocation problem

3.1.1. Input

In the GP allocation problem we are given a set of patients $P = \{p_1, p_2, ..., p_n\}$ and a set of doctors $D = \{d_1, d_2, ..., d_m\}$. We are also given the current and preferred GP assignments, for each patient:

- $D_{\text{cur}}[i]$ denotes the index of the current doctor assigned to patient p_i (or $\perp$ if unassigned)
- $D_{\text{pref}}[i]$ denotes the index of the preferred doctor for patient p_i

In addition we are given a priority function $R : P \to \mathbb{N}$, mapping some positive integer to each patient, with higher numbers indicating a higher priority to help this patient.

3.1.2. Feasible solution

A feasible solution for the GP allocation problem is a subset of patients $S \subseteq P$ such that the directed graph

$$G_S = (S, E_S), E_S = \left\{(a, b) \mid a, b \in S, D_{\text{pref}}[\text{idx}(a)] = D_{\text{cur}}[\text{idx}(b)]\right\} \tag{4}$$

consists of a vertex-disjoint union of directed cycles that cover all vertices in S. This means that S is a set of patients that can all get their preferred doctor if we allow for simultanous exchanges following cycles.

3.1.3. Optimization criterion

Next we can start defining variants of feasible solutions where we want to maximize some *score*. Examples of such a score could be to find a feasible solution with many patients or something based on priority. We might want a solution that exchanges as many patients as possible, exchanges patients with the highest priority or some mix of these.

First we define our general ordering

Definition 3.1.3.1 (Optimization criterion): An ordering $\succ$ over feasible solutions. A solution is optimal if it is maximal under $\succ$.

Build on this to create our three main variants of scoring functions.

3.1.3.1. Lexicographic maximization by priority

Definition 3.1.3.1.1 (Characteristic vector): Let patients be ordered by priority: $p_1 \geq p_2 \geq ... \geq p_n$ where p_1 has the highest priority. Given a feasible solution $S \subseteq P$, the **characteristic vector** is:

$$\chi(S) = (b_1, b_2, ..., b_n) \in \{0,1\}^n, \quad b_i = \begin{cases} 1 \text{ if } p_i \in S \\ 0 \text{ otherwise} \end{cases} \tag{5}$$

Definition 3.1.3.1.2 (Lexicographic ordering by priority):

$$S \underset{\text{lex}}{\succ} S' \quad \text{iff} \quad \chi(S) \text{ is lexicographically greater than } \chi(S') \tag{6}$$

That is, at the first index i where S and S' differ, $\chi(S)_i = 1$ and $\chi(S')_i = 0$.

Intuitively, $\chi(S)$ is a binary number whose most significant non-zero bit corresponds to the highest-priority patient that gets helped. The optimal solution is then the one that maximises this binary number: always satisfying the highest-priority patient it can, then subject to that the next highest, and so on.

Example (Ordering two solutions lexicographically by priority):

We use the example graph in Figure 5 as the graph to create solutions from.

Given solutions
1. $S = \{p_4, p_5\}$ represents the subgraph G_S containing the cycle $p_4 \to p_5 \to p_4$
2. $S' = \{p_1, p_2, p_3\}$ represents the subgraph $G_{S'}$ containing the cycle $p_1 \to p_2 \to p_3 \to p_1$

For simplicity's sake we say that $R(p_i) = i$, so p_5 has the highest priority.

Ordering all five patients by priority gives $p^{(1)} = p_5, p^{(2)} = p_4, ..., p^{(5)} = p_1$. Then:

$$\chi(S) = (1,1,0,0,0) \quad \chi(S') = (0,0,1,1,1) \tag{7}$$

At the first differing position ($i = 1$), $\chi(S)_1 = 1 > 0 = \chi(S')_1$, so $S \underset{\text{lex}}{\succ} S'$.

So for our first variant we want to find the maximal solution under the $\underset{\text{lex}}{\succ}$ ordering. This means always prioritizing patients with highest priority.

3.1.3.2. Maximizing cardinality

Another natural variant is to find a solution with the largest cardinality, e.g. a solution that contains the most patients.

Definition 3.1.3.2.1 (Ordering by cardinality):

$$S \underset{\text{size}}{\succ} S' \text{ iff } |S| > |S'| \tag{8}$$

This optimal solution will then be one that exchanges the most patients.

Example (Ordering two solutions by cardinality):

Using the same solutions as in the previous example:
1. $S = \{p_4, p_5\}$ represents the subgraph G_S containing the cycle $p_4 \to p_5 \to p_4$
2. $S' = \{p_1, p_2, p_3\}$ represents the subgraph $G_{\{S'\}}$ containing the cycle $p_1 \to p_2 \to p_3 \to p_1$

Under $\underset{\text{size}}{\succ}$: $|S| = 2$ and $|S'| = 3$, so $S' \underset{\text{size}}{\succ} S$. However the solution is $\{p_1, p_2, p_3, p_4, p_5\}$, the union of both cycles.

Notice that the same two sets are reversibly ordered under the two criteria: $S \underset{\text{lex}}{\succ} S'$ (because S contains the highest-priority patient p_5) but $S' \underset{\text{size}}{\succ} S$ (because S' has more patients).

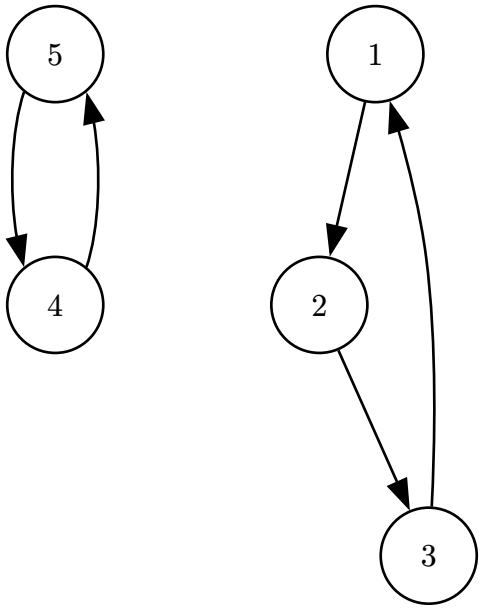

Figure 5: An example graph G.

3.1.3.3. Maximizing utility

Finally we formulate an ordering that is a combination of $\underset{\text{lex}}{\succ}$ and $\underset{\text{size}}{\succ}$. We define the total utility to be the sum of the priorities of the patients in S.

We recall that $R : P \to \mathbb{N}$ is the function mapping a patient to a priority.

Definition 3.1.3.3.1 (Total utility function):

$$U(S) = \sum_{p \in S} R(p) \tag{9}$$

Definition 3.1.3.3.2 (Ordering by total utility):

$$S \underset{\text{util}}{\succ} S' \text{ iff } U(S) > U(S') \tag{10}$$

This means that even though solution S' contains some high priority patients, if S contains enough lower priority patients then S can still be a higher ranked solution under $\underset{\text{util}}{\succ}$.

Example (Ordering two solutions by total utility):

We use the following solutions from Figure 6:
1. $S = \{1, 3, 4\}$ representing the cycle $p_1 \to p_3 \to p_4 \to p_1$
2. $S' = \{5, 2\}$ representing the cycle $p_5 \to p_2 \to p_5$

This gives the following total utilities:
$U(S) = 1 + 3 + 4 = 8$
$U(S') = 2 + 5 = 7$

It follows that $S \underset{\text{util}}{\succ} S'$. Here we see that the solution with more patients with lower maximum priority has a higher score than another with greater maximum priority.

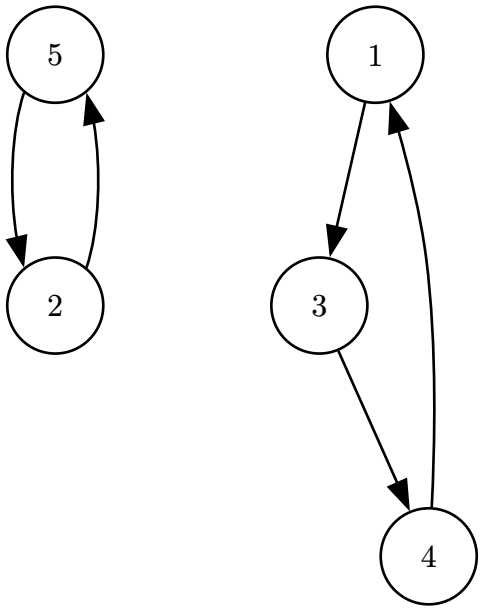

Figure 6: An example graph G.

3.1.4. Priority function

The priority function has quite a large effect on the final solution. The priority chosen will largely decide whether that patient will be in the final solution or not. This is why its important to discuss what effects the function can have on solutions when we are maximizing the ordering $\underset{\text{util}}{\succ}$ or $\underset{\text{lex}}{\succ}$.

If we make the priority function exponential such that $R(a) = 2^a$, then using the ordering $\underset{\text{util}}{\succ}$ becomes the same as $\underset{\text{lex}}{\succ}$. This is because 2^a is larger than the sum of all lower priorities below it. If patient a has priority 2^a, then no group of patients with lower priority can have a total

priority that reaches 2^a, since their priorities sum to at most $2^a - 1$. So a solution maximizing $\underset{\text{util}}{\succ}$ will always pick the higher priority patient first, which is exactly what $\underset{\text{lex}}{\succ}$ does.

While this ordering is in a sense the most fair, since we never let a patient with lower priority switch before one with greater, it might not always be the best for the collective good, since a high priority patient might block several lower priority patients from switching. One solution to this could be a priority function where $R(a)$ depends on how long patient a has been waiting for a switch. Then if we have the choice between patient p_i who has waited 30 days, $R(p_i) = 30$, and four patients who have waited 40 days in total, an algorithm maximizing $\underset{\text{util}}{\succ}$ will choose the four patients.

We can also use a priority function that sits between these two extremes. Instead of a fixed base we use a base k with $1 \leq k \leq 2$ and set

$$R(a) = k^{\text{days patient } a \text{ has been waiting}}. \tag{11}$$

The base k controls how much higher priority patients are favored. At $k = 1$ every patient gets priority 1, no matter how long they have waited. The total utility $U(S)$ is then just the number of patients in S, so maximizing $\underset{\text{util}}{\succ}$ becomes the same as maximizing $\underset{\text{size}}{\succ}$. At $k = 2$ the priorities grow fast enough that a higher priority patient is worth more than any group of lower priority patients below them, so $\underset{\text{util}}{\succ}$ becomes the same as $\underset{\text{lex}}{\succ}$, as shown above. A base between the two gives a mix, and we can tune it depending on how much we want to favor patients who have waited longer.

So the cardinality and lexicographic orderings are not separate cases but the two endpoints of the utility ordering. This is why our exact algorithm for $\underset{\text{util}}{\succ}$ also solves $\underset{\text{size}}{\succ}$, by giving it the priority function with $k = 1$. The lexicographic case at $k = 2$ is different in practice. The priorities 2^a grow very large, and the runtime of cycle cancelling depends on the size of the costs, so running the utility algorithm with $k = 2$ would be slow. For this reason we treat strict lexicographic priority as its own case with its own algorithm, which we describe in the next chapter.

4. Implementation

In this chapter we look at each algorithm we have implemented, how we implemented them, why and runtime analysis. We have in total implemented four algorithms, Greedy DFS, Cycle cancelling for cardinality, utility and for strict priority. For the cycle cancelling algorithms we also give, or refer to existing, proofs for why they are exact and why they run in polynomial time.

4.1. Different graph representations used

In the algorithms we use different graph representations, so we start by defining the different graph representations of *The GP allocation problem*.

4.1.1. Patient and GP graph

First we have the easiest graph containing both patients and GPs as vertices. The edges are directed from a patient to a GP and from a GP to a patient. Edges never go patient $\rightarrow$ patient or GP $\rightarrow$ GP. An edge patient $\rightarrow$ GP symbolizes that the patient has that GP as its preferred GP, and GP $\rightarrow$ patient symbolises that the GP currently has that patient as one of his or hers current patients. Observe that this graph is bipartite as we have patients on one side and GPs on the other. Now the formal definition of our graph. Let $I = \{0, ..., |P| - 1\}$. Then:

$$\begin{gathered} G = (V, E), \quad V = P \cup D \\ E = \left\{ \left(p_i, D_{\text{pref}[i]} \right) \mid i \in I \right\} \cup \left\{ \left(D_{\text{cur}[i]}, p_i \right) \mid i \in I \right\} \end{gathered} \tag{12}$$

4.1.2. GP graph collapsed edges

In this weighted graph we condense the problem to only have GPs as nodes and edges between GPs. An edge GP a $\rightarrow$ GP b symbolises that there exists a patient that wants to switch from GP a to GP b, or that the patient currently has GP a and has GP b as preferred. The capacity of an edge indicates the number of patients wanting that switch, while the cost is –1. We define our graph as:

$$\begin{gathered} G = (V, E), \quad V = D \\ E = \left\{ (a, b) \mid \exists i \in I \ \text{ s.t. } \ D_{\text{cur}[i]} = a \wedge D_{\text{pref}[i]} = b \right\} \\ u(a, b) = |\left\{ i \in I \mid D_{\text{cur}[i]} = a \wedge D_{\text{pref}[i]} = b \right\}| \\ c(a, b) = -1 \end{gathered} \tag{13}$$

4.1.3. GP graph priority weighted

This weighted graph is much like the GP graph collapsed edges, but while that graph representation focuses on number of patients wanting a switch this graph focuses on the priority of each patient. Instead of collapsing all preferences that are equal here we make each preference into an edge and weight it with the priority of that patient.

$$
\begin{aligned}
G = (V, E), \quad V = D \\
E = \left\{ \left(D_{\text{cur}[i]}, D_{\text{pref}[i]}, i \right) \mid i \in I \right\} \\
u(a, b, i) = 1 \\
c(a, b, i) = -R(P_i)
\end{aligned}
\tag{14}
$$

4.2. Greedy DFS

We first consider a greedy approach inspired by the Top Trading Cycles implementation of Huitfeldt et al. Their algorithm preserves TTC properties at each round by restricting each GP node to a single outgoing edge [4]. We relax this constraint, allowing each GP to keep outgoing edges to all of its current patients at the same time, and resolve ties by patient priority.

This algorithm runs on the Patient and GP graph defined in Section 4.1.1. Recall that this graph has both patients and GPs as vertices, with an edge $p_i \to D_{\text{pref}[i]}$ from each patient to their preferred GP, and an edge $D_{\text{cur}[i]} \to p_i$ from each GP to every patient it currently has. A directed cycle in this graph alternates between patient and GP nodes and corresponds to a valid exchange. Each patient in the cycle moves to their preferred GP, and each GP loses one patient and gains one.

We are also given the priority function $R : P \to \mathbb{N}$, where a higher number means higher priority. The algorithm processes patients in decreasing priority order. For each patient it runs a depth first search that tries to find a cycle through that patient. When the search reaches a GP node with several current patients, it explores the highest priority patient first.

GreedyDFS(G, R)

```
resolved = []
sorted = patients sorted by R in decreasing order
for each p ∈ sorted do
    cycle = dfsFindCycle(G, R, p)
    if cycle ≠ none then
        for each q ∈ cycle do
            resolved.push(q)
        end
    end
end
return resolved
```

The depth first search dfsFindCycle(G, R, p) starts at patient p and follows edges through the graph. At a patient node it follows the one edge to that patient's preferred GP. At a GP node it has several edges to choose from, one to each current patient, and it tries them in decreasing priority order. The search returns a cycle if it reaches p again, and returns nothing if it cannot. When a cycle is found, every patient in it is added to the resolved set and is not considered again.

The greedy rule makes sure high priority patients are preferred when forming cycles, but it does not guarantee a globally optimal solution. The following example shows how the greedy choice can be locally motivated but globally suboptimal.

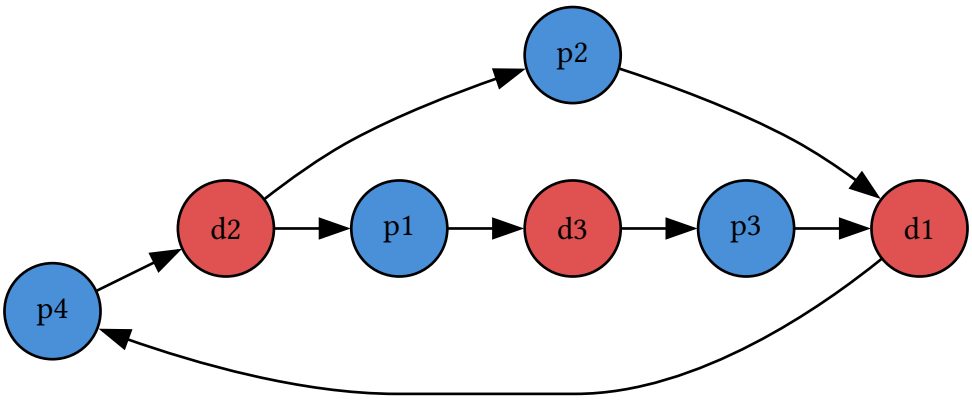

Figure 7: Example graph G where Greedy DFS would choose suboptimal solution. px means patient x and has priority x.

In Figure 7 each patient p_x has priority x. The search begins at p_4, the highest priority patient, and follows the edge to d_2. At d_2 it chooses p_2 over p_1 since $R(p_2) > R(p_1)$. The resulting cycle $p_4 \to d_2 \to p_2 \to d_1 \to p_4$ is committed, which leaves p_1 and p_3 unsatisfied with no further cycles remaining.

Had the search chosen p_1 at d_2 instead, it would have found the longer cycle $p_4 \to d_2 \to p_1 \to d_3 \to p_3 \to d_1 \to p_4$, which satisfies three patients. The greedy choice at d_2 was locally motivated by priority but globally suboptimal. This motivates the exact algorithms in the following sections.

4.3. Cycle cancelling for cardinality

This algorithm finds the solution maximal under $\succ_{\text{size}}$. We use the GP graph with collapsed edges representation defined in Section 4.1.2. This way each edge (v, w, i) represents patients wanting to switch from GP v to w, the cost is -1 and the capacity is the number of patients wanting that switch. We then want to find a circulation f with minmial cost, we use cycle cancelling to do this. The following is the implementation of *Cycle Cancelling for cardinality* using *Cycle Cancelling* as defined in section Section 2.3.

CycleCancellingCardinality(G, P)

```
1 resolved = ∅
2 f = zero circulation on G
3 f* = CycleCancelling(G, f)
4 for each edge (v, w, i) ∈ G do
5 |  if f*(v, w, i) > 0 then
6 |  |  let S = {p ∈ P | D_cur[p] = v ∧ D_pref[p] = w}
7 |  |  pick any f*(v, w, i) patients from S and add them to resolved
8 |  end
9 end
```

CycleCancellingCardinality(G, P)

```
10 return resolved
```

Because we choose edges based only on the capacity, e.g the number of patients wanting that switch, we do not know which k patients to actually resolve when we find out that k patients can switch on an edge. This choice is arbitrary since all patients in this representation are equally important and it does not effect the cardinality of the solution. While we could choose random patients, the realistic choice would be to choose the k patients who have the highest priority among those who want the swap.

4.3.1. Runtime

The runtime of *Cycle Cancelling* as mentioned is $O(nm^2CU)$ making the algorithm pseudo polynomial based on C, the max cost, and U, the max capacity. In the GP graph with collapsed edges the max cost, C, is one. While the max capacity, U, is equal to the number of patients. So the runtime of *Cycle Cancelling for cardinality* is polynomial in terms of the input G, P as the runtime is $O(nm^2\ |P|)$

Because of the *Negative Cycle Optimaily Theorem* the *Cycle Cancelling* will not finish until there are no more negative cycles in the residual graph. Since the cost of each edge in the original graph is −1 and *Cycle Cancelling* minimizes the total cost, the algorithm finds the solution with the highest cardinality.

4.4. Cycle Cancelling for utility

This algorithm finds the solution maximal under $\underset{\text{util}}{\succ}$. We use the GP graph priority weighted representation defined in Section 4.1.3. So each patient, i, is an edge in G, the capacity is one while the cost is $-R(i)$. The algorithm then proceeds as follows:

CycleCancellingUtility(G, P)

```
1 resolved = ∅
2 f = zero circulation on G
3 f* = CycleCancelling(G, f)
4 for each edge (v, w, i) ∈ G do
5 |  if f*(v, w, i) > 0 then
6 |  |  Add P[i] to resolved
7 |  end
8 end
9 return resolved
```

4.4.1. Runtime

The runtime differs from the cardinality case because the graph parameters are different. The maximum capacity U is now 1, since every patient is its own capacity-1 arc, while the

maximum cost C is $\max R(p)$. The bound $O(nm^2CU)$ becomes $O(nm^2 \max R(p))$. This is polynomial as long as the priorities are polynomially bounded in the input size.

For the strict lexicographic case, the priority function is $R(p_i) = 2^i$, which grows exponentially in the number of patients. With that priority function C is exponential in n, so running this algorithm on it would no longer be polynomial. This is why we treat the strict lexicographic case separately and give it its own algorithm.

4.5. Cycle Cancelling for strict priority

The *Cycle Cancelling for strict priority* finds the solution maximal under $\underset{\text{lex}}{\succ}$. It uses the GP graph priority weighted representation defined in Section 4.1.3 and uses *Cycle Cancelling*, but modifies it. First we loop through patients in descending order of priority, if we find any cycle with this patient in the residual graph we cancel it and add the patient to the solution. If the patient already has flow from being in a cycle with another patient with greater priority, then we just remove it from the graph. This way the highest priority patient is either in a cycle or not, the patient is removed from the graph at the end either way. The algorithms is as follows:

CycleCancellingStrictPriority(G, R, P)

```
resolved = ∅
f = zero circulation on G
sorted = P sorted by R in decreasing order
for each p ∈ sorted do
    i = idx(p)
    e = (D_curr[i], D_pref[i], i)
    if f(e) > 0 then
        Add p to resolved
        Delete e from G
    end
    if ∃ negative residual cycle c ∈ G(f) and e ∈ c then
        Cancel c
        Add p to resolved
    end
    Delete e from G
end
return resolved
```

The final solution is the maximal solution under $\underset{\text{lex}}{\succ}$ this is because we process patients starting from those with greatest priority. Because one high priority patient is "better" in terms of $\underset{\text{lex}}{\succ}$ than all patients with lesser priority, if we find a cycle that allows that high priority patient to be resolved we must use it. Then by removing the edge from the graph we do not allow that edge to be "undoed". If we do not find a cycle containing the high priority patients edge then there is no way to circulate flow along this edge e.g it is not possible to satisfy that patient.

4.5.1. Runtime

As we loop over all patients we have |P| iterations. For each iteration we search for a negative residual cycle, recall we can use Bellman-Ford in $O(nm)$ time for this. So the *Cycle Cancelling for strict priority* has a runtime of $O(nm\ |P|)$ and is not dependent on the maximum priority as in the *Cycle Cancelling for utility*.

5. Experimental Results

5.1. Experimental Setup

5.2. Performance Comparison

5.3. Impact of Priority Strategies

6. Conclusion

6.1. Summary

6.2. Future Work

Bibliography

[1] Legelisten, "Med rett til å bytte – Ventetider for å bytte fastlege i Norge." Accessed: Apr. 08, 2026. [Online]. Available: https://legelisten.s3.eu-west-1.amazonaws.com/blog-files/Med+rett+til+%C3%A5+bytte+-+Ventetider+for+%C3%A5+bytte+fastlege+i+Norge.pdf

[2] Helfo, "Fastlegestatistikk." Accessed: Apr. 08, 2026. [Online]. Available: https://www.helfo.no/Fastlegeordninga/fastlegestatistikk

[3] Statistisk sentralbyrå, "Fastlegelister og fastlegekonsultasjoner, etter år og region (12005)." Accessed: Apr. 08, 2026. [Online]. Available: https://www.ssb.no/statbank/table/12005

[4] I. Huitfeldt, V. Marone, and D. C. Waldinger, "Designing Dynamic Reassignment Mechanisms: Evidence from GP Allocation," Working Paper 32458, May 2024. doi: 10.3386/w32458.

[5] L. Shapley and H. Scarf, "On cores and indivisibility," *Journal of Mathematical Economics*, vol. 1, no. 1, pp. 23–37, 1974, doi: https://doi.org/10.1016/0304-4068(74)90033-0.

[6] D. J. Abraham, A. Blum, and T. Sandholm, "Clearing algorithms for barter exchange markets: enabling nationwide kidney exchanges," pp. 295–304, 2007, doi: 10.1145/1250910.1250954.

[7] Wikipedia contributors, "Optimal kidney exchange — Wikipedia, The Free Encyclopedia." Accessed: May 05, 2026. [Online]. Available: https://en.wikipedia.org/w/index.php?title=Optimal_kidney_exchange&oldid=1351916222

[8] D. Gale and L. S. Shapley, "College Admissions and the Stability of Marriage," *The American Mathematical Monthly*, vol. 69, no. 1, pp. 9–15, 1962, Accessed: May 06, 2026. [Online]. Available: http://www.jstor.org/stable/2312726

[9] Wikipedia contributors, "Top trading cycle — Wikipedia, The Free Encyclopedia." Accessed: May 08, 2026. [Online]. Available: https://en.wikipedia.org/w/index.php?title=Top_trading_cycle&oldid=1344910705

[10] A. E. Roth, "Incentive compatibility in a market with indivisible goods," *Economics Letters*, vol. 9, no. 2, pp. 127–132, 1982, doi: https://doi.org/10.1016/0165-1765(82)90003-9.

[11] R. K. Ahuja, T. L. Magnanti, and J. B. Orlin, *Network Flows: Theory, Algorithms, and Applications*. Prentice Hall, 1993.

[12] Wikipedia contributors, "Bellman–Ford algorithm — Wikipedia, The Free Encyclopedia." [Online]. Available: https://en.wikipedia.org/w/index.php?title=Bellman%E2%80%93Ford_algorithm&oldid=1351231946

[13] A. V. Goldberg and R. E. Tarjan, "Finding minimum-cost circulations by canceling negative cycles," *J. ACM*, vol. 36, no. 4, pp. 873–886, Oct. 1989, doi: 10.1145/76359.76368.

Index of Figures